THEME: VIS + AI · INFORMATION VISUALIZATION TERM PROJECT PROPOSAL

LagLens: What Does a Robot Do While Its Policy Is Still Thinking?

A visual analytics tool for robot policies that plan in chunks and need time to think

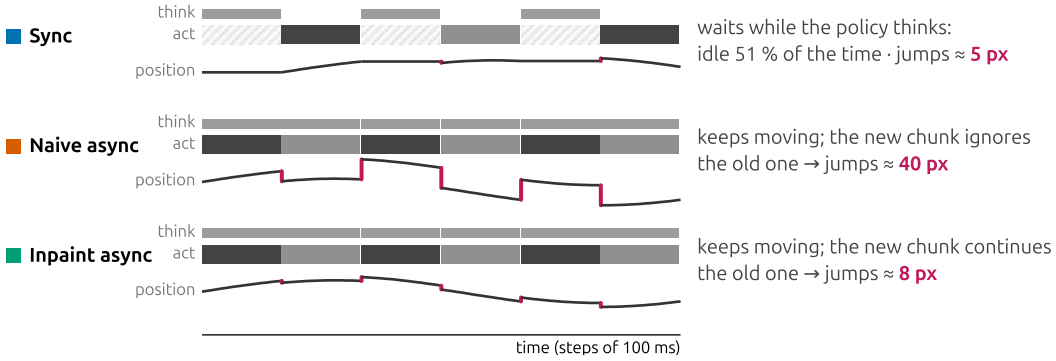
How such a robot works

1. **Plan.** A robot **policy** (a neural network) looks at the camera once and plans its next 16 moves: an **action chunk**.
2. **Act.** The robot performs the first *h* moves of the chunk without looking again, then needs a new chunk.
3. **Think.** Computing a chunk takes time *d* (**inference latency**, here 0–400 ms). What should the robot do meanwhile? (right)
   • **Sync**: **wait**. Fine while the world waits too, but the robot stands idle, and anything that moves gets away meanwhile.
   • **Async**: **keep moving** on the old chunk. No waiting, but the scene changes while the policy thinks, so the new chunk is made for a scene that no longer exists (an **outdated plan**) and may not continue the current motion: the robot **jumps** when it switches (a **chunk-boundary discontinuity**). **Naive async** just swaps the chunks; **inpaint async** computes the new chunk so that it continues the old one.

The problem. Given their latency *d*, engineers pick a strategy and *h* by comparing one success-rate number per combination, e.g. “d = 4, h = 4: sync 13 % vs. inpaint async 67 %”. That number cannot show **which** start states failed, **when**, or **why** (idling, outdated plans or jumps). We found no visualization tool for this.

Our idea: same start, different schedule. We run a policy from the **same 30 start states** under **14 schedules** (3 strategies × $d \in \{0, 2, 4\}$ × $h \in \{4, 8\}$) and record every planned chunk, on **two tasks**: **PushT** (hard; the block waits while the robot thinks) and **Catch the ball** (the ball keeps rolling while the robot thinks). Any two schedules can then be compared start by start and replayed move by move.

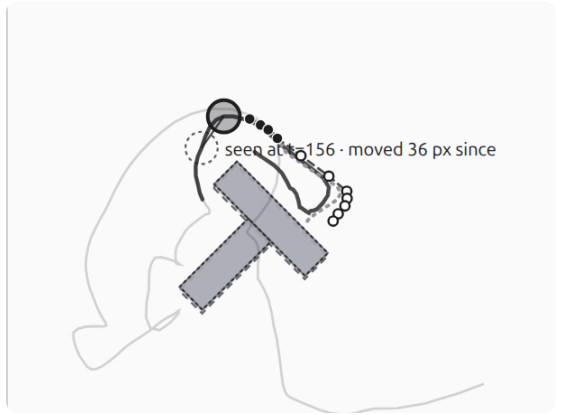
The three strategies over time (d = 4, h = 4)



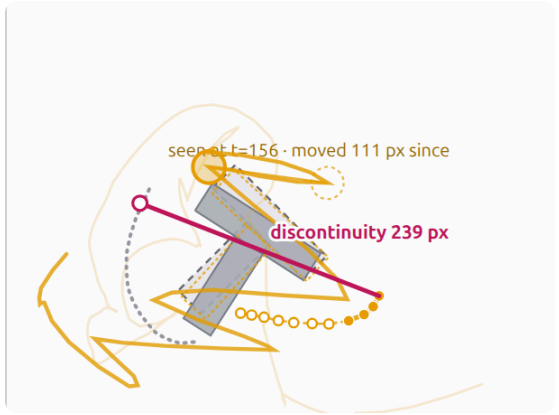
think = policy computing a chunk · act = robot executing it (hatched = standing still) · position = where the robot is (schematic) · red step = jump at a chunk switch, drawn to scale: mean of our 30 start states

A real case from our data: same start (#1017), d = 4 (400 ms), h = 4

Task: push the grey T-block onto the dashed T-goal (top view). Only the strategy differs.



Inpaint async ✓ success



Naive async ✗ failure: a 239 px jump at a switch knocks the block out of the goal

Our own recorded runs of two robot policies on two tasks

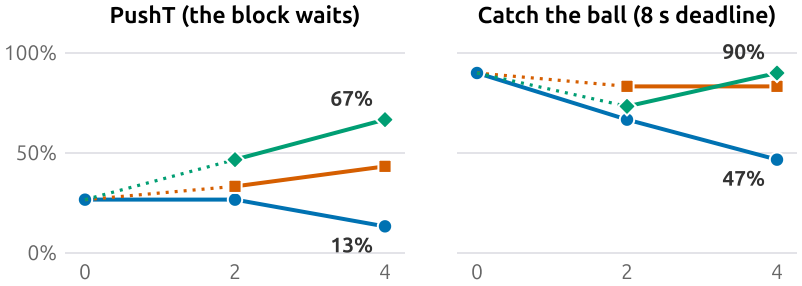
Same round pusher in both simulated tasks (top view, 512 × 512 px; a move = target position of the pusher). **PushT** (hard): push a T-shaped block onto a fixed T-shaped goal; pretrained Diffusion Policy `lerobot/diffusion_pusht` (last 2 camera frames + pusher positions). **Catch the ball** (ours): touch a ball that rolls and bounces (42–100 px/s, new speed and direction per start state) within 8 s; a small Diffusion Policy we trained on scripted interception demonstrations (pusher and ball positions). Both plan chunks of 16 moves from random noise, so re-running a schedule can change an outcome; a *noise replicate* measures how often. Per task: 30 start states × (14 schedules + 1 replicate) = 450, in total **900 episodes** (recorded runs); every planned chunk and every executed move is logged (47 KB CSV + 21 MB JSON).

Attribute (episode table, 900 items × 23 attributes + task)	Type · ordering direction	Order / range / levels	Meaning
task KEY	nominal (no order)	PushT · Catch the ball	what the robot must do (one data folder per task)
strategy KEY	nominal (no order)	sync · naive · inpaint	what the robot does while a chunk is computed
latency d KEY	ordinal (ratio, steps)	0 < 2 < 4 steps (0–400 ms)	inference delay; d = 0 only for sync
horizon h KEY	ordinal (ratio, steps)	4 < 8	moves executed from each chunk before switching
replicate · seed KEY	nominal (binary · ID)	0 · 1 | 1000–1029	1 = sync d0 h8 re-run with new noise | start state, same in every schedule
schedule	nominal (ID)	15 levels, e.g. `sync_d4_h4`	name for one (strategy, d, h, replicate) combination; not an extra key
success	nominal (binary)	0 = fail, 1 = success	PushT: coverage > 95 % within 300 steps · Catch: pusher touches the ball within 80 steps
coverage_final / max · dist_final	quant. ratio, ↑ · ↓	0.00–0.99 / 0.00–0.99 · 32–426 px	PushT: block–goal overlap at the end / best · Catch: pusher–ball distance at the end
ball_speed	quant. ratio	42–100 px/s	Catch only: speed of the ball in this start state
steps_to_success	quant. ratio, ↓	74–300 · 12–80 (empty if failed)	time to finish (PushT · Catch)
jump_mean / max	quant. ratio (px), ↓	0.3–381.9 / 1–949	jump at a chunk switch: gap between the new chunk's first move and the old chunk's next one
think_motion_mean / max	quant. ratio (px), ↓	0.0–227.8 / 0–499	how far the robot moved while the policy was thinking (change the new chunk did not see)
staleness_mean	quant. ratio (steps)	1.4–7.5	age of the picture behind each move (≈ d + (h−1)/2, equal for all strategies)
idle_frac	quant. ratio, ↓	0.00–0.60	share of steps with nothing to execute
jerk · inner_step · path_len	quant. ratio (px)	1.3–232.9 · 2.9–70.6 · 111–6158	smoothness (2nd difference of moves) · move size inside a chunk · distance travelled
n_inferences · n_switches	quant. ratio (count)	2–75 · 1–74	# chunks computed · # chunk switches

Ordering direction: ↑ higher is better · ↓ lower is better · no arrow = no preferred direction.

Dataset type: multidimensional **table** of episodes, linked to two **temporal** tables and a **spatial** field. **Data types**: **items** (episodes, steps, chunks) with **attributes**; **positions** (x, y). **Step table** (131,872 items; key t, ordinal, 10 Hz): pusher, block (PushT) or ball (Catch) and move x, y (interval, px); block θ (cyclic); chunk id (nominal); index j (ordinal, 0–15); age (ratio, steps); held (binary, idle); coverage or distance (ratio). **Chunk table** (21,689 items): picture time and ready time (ordinal, step), executed window (step range), 16 planned moves (interval, px). **Keys** (marked KEY): task, strategy, d, h, replicate → schedule; × seed → episode; × t → step; × chunk id → chunk. **Values**: all attributes without KEY (outcomes, costs, counts; per step positions, move, age, held).

First look: when does acting while thinking pay off?



Sync ● **Naive** ■ **Inpaint** ◆ · success rate, real data. In **Catch** the ball rolls on while sync waits: at d = 4, h = 4 sync 47 %, naive 83 %, inpaint 90 % (inpaint +13 / −0 start states, naive +14 / −3; a re-run flips 4), and async catches sooner (27 vs. 44 steps). In **PushT** async also wins at d = 4; the tool shows why (next slides).

PushT: pretrained policy `lerobot/diffusion_pusht` (LeRobot), simulator gym-pusht · Catch: our task and policy (`scripts/catch_task.py`); ball size and deadline set so that sync *without* latency succeeds ≈ 90 % (separate seeds) before any latency was tested

Two people who must decide how a policy is executed

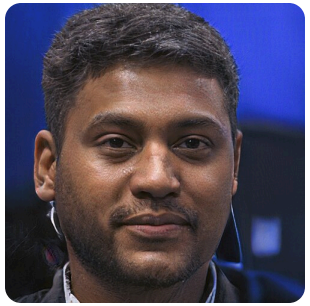


PRIMARY PERSONA

Mara Jensen

28 · 4th-year PhD student, robot learning lab · Copenhagen, Denmark

- **Context**: trains diffusion / flow-matching policies (generative models that output action chunks); deploys them on a real arm while the GPU server sits across the building (60–250 ms Wi-Fi latency).
- **Goals**: pick an execution strategy and horizon that keep success high at the latency she actually has; explain *why* in a paper.
- **Needs**: per-episode evidence of what failed, aligned with what the policy planned at that moment.
- **Pain points**: only success-rate tables and video; cannot tell whether a failure came from waiting, an outdated plan or a jump; re-running real robot trials is costly; matplotlib debug plots show one episode at a time.



SECONDARY PERSONA

Arjun Mehta

37 · ML platform engineer, warehouse-robotics start-up · Seoul (Pangyo), Korea

- **Context**: owns the policy-serving software (the robot sends camera frames to a GPU server, which returns action chunks); does not train models. Robots pick parcels from moving conveyors, so objects do not wait for inference.
- **Goals**: set a latency budget (the largest inference delay a robot may tolerate) per robot and a safe default strategy; know when to fall back to sync.
- **Needs**: an overview of how success degrades with latency for each strategy, with confidence that it is not a few lucky seeds.
- **Pain points**: research papers report curves on other robots; no shared vocabulary with researchers about “why it jitters”.

Difference	Mara (primary)	Arjun (secondary)
Expertise	Policy internals (diffusion, chunking)	Systems, networking, latency
Granularity of questions	Single episodes, single chunk switches	Schedules and aggregate trends
Main view used	Workspace replay + execution timeline, seed shifts (T3, T4)	Latency-response curves + paired seed table (T1, T2)
Decision made	Which strategy / horizon to propose and why	Latency budget and fallback rule

Portraits: StyleGAN-generated, public domain (Wikimedia Commons “Woman 1.jpg”, “Man 2.jpg”); the people do not exist. Personas are fictional.

Four tasks, from all schedules down to a single chunk switch

#	Action + target	Concrete question (Mara)	Why	Expected outcome
T1	**Compare → trends** of success and of each strategy's cost (idle time, outdated plans, jumps) across schedules	"How fast does success fall with latency for each strategy and horizon, when the world waits (PushT) and when it moves (Catch)?"	Choose a candidate schedule for the latency the lab actually has.	A shortlist of robust schedules and a latency budget per strategy.
T2	**Locate → outliers**: initial states whose outcome **flips** between a baseline and a candidate schedule	"Going from sync d=0 to naive async d=4, *which* seeds start failing, and which are rescued?"	Aggregate rates hide which situations break; paired seeds isolate the schedule's effect.	A small set of fragile seeds to inspect (and to add to a regression test set).
T3	**Identify → features** in time & space: where idling, outdated plans and jumps occur in a flipped episode	"At which step does the candidate diverge from the baseline, and did the robot wait, act on an outdated plan, or jump?"	Diagnose the mechanism of failure, not just its rate.	A causal hypothesis, e.g. "a 239 px jump knocks the block out of the goal" (PushT) or "sync waits while the ball rolls away" (Catch).
T4	**Discover → correlation** between jumps at chunk switches (boundary discontinuities) and task outcome (coverage, distance to the ball) over paired episodes	"When a seed moves from the baseline to the candidate, do larger discontinuities go with worse outcomes? Does inpainting shrink the discontinuities?"	If jumps are what breaks episodes, a strategy that shrinks them should rescue seeds; check it on our policy.	Evidence for or against a strategy (direction and size of shift per seed).

Arjun mainly needs **T1** (latency budget) and **T2** (is the drop driven by a few seeds?). **Mara** chains T1 → T2 → T3 → T4 in one session: overview first, then drill down to a single chunk switch (details on demand). Both run the same tasks on **Catch the ball** first, where the cost of waiting is obvious, and then on **PushT**, where the costs of waiting, outdated plans and jumps compete.

Each view answers one task with the most effective idiom

View	Task	Idiom & why	Interaction
A · Latency Response	T1	*"How fast does each strategy get worse as the policy gets slower?"* **Line chart small multiples**: one panel per horizon h, x = latency d on its true scale, y = metric, strategies **superposed** (hue + shape). Position on a common scale beats the luminance of a heatmap; superposition for direct sync-vs-async comparison, juxtaposition across h. **95 % interval** lines instead of bars; hover = paired difference to the baseline.	Click mark = candidate, shift-click = baseline; **one question bar**: Success · Idle (cost of sync) · Outdated (cost of async) · Discontinuity (cost of naive async), drives A *and* B; recomputes on seeds selected in B.
B · Paired Seed Table	T2, T4	*"Which start states flip between baseline and candidate, and is that more than chance?"* **Composite Table-Lens view**: one row per seed shared by (left) an outcome matrix — filled = success, baseline column black, candidate column orange — and (right) **dumbbells** baseline → candidate of the attribute that belongs to A's question. Rows are **grouped by baseline→candidate change** (spatial region), so flips break their band's pattern; a **re-run** column shows chance flips (PushT 8/30).	No controls of its own. Click column header = candidate; click cell = open episode; **brush** dumbbells or click a band → **cross-filter** A.
C · Workspace Replay	T3	*"Watch both runs side by side: what was planned, what was done, where it jumped."* Juxtaposed baseline | candidate maps at identical scale (true geometry). Overlays: current chunk, executed (solid) and thrown away (dashed); next chunk being computed (dotted); latest **discontinuity** (magenta); **what the policy saw** (dotted outline of block or ball & pusher in the chunk's picture, arrow to the robot now: the gap = how outdated the plan is).	Play / scrub / ← → (**navigate** t); hover a planned point → when planned / executed; click → jump.
D · Execution Timeline	T3	*"When was the policy thinking, which plan did each move come from, where were the jumps?"* **Gantt lanes** on a shared linear time axis: per run a **think** track (inference) and an **act** track (chunk spans, hatched = idle), a **motion** track (movement per step, jumps as magenta bars on the same scale), coverage (PushT) or ball distance (Catch) below. Sync shows think/act *alternating*, async *overlapping*. The chunk at the cursor is traced **picture ○ → think → act**, with a 16-cell strip of its planned moves: skipped while thinking · executed · discarded.	**Brush** a time window → highlighted path in view C; hover chunk → its plan in view C; click → cursor.

Why this is not a generic dashboard: C and D trace every executed move back to the plan (chunk) and the camera frame it came from, which no chart tool offers; A and B compare two schedules *start state by start state* against a chance baseline (re-run), not just averages. **Two tasks, one tool**: a task switch in the header shows the same four views for **Catch the ball**, where async's advantage is plain to see (sync stands still while the ball rolls away), next to **PushT**, where the costs compete. **Less is more**: the first draft had 5 views and ~10 hues; following parsimony and consistency we merged two views, gave every hue one meaning and cut ten controls to **one question bar**.

The most important attribute gets the most effective channel

Attribute (type)	View	Mark	Channel	Justification
success rate, idle share, motion while thinking, boundary discontinuity (ratio)	A	point + line	position y (common scale)	best magnitude channel; slope = degradation
latency d (ratio)	A	—	position x (true spacing)	100 ms steps, not ordinal bins
horizon h (ordinal)	A	panel	spatial region (small multiples)	juxtaposed, shared y
strategy (nominal, 3)	A, B	line, point	hue + shape (●■◆)	redundant: never hue alone
uncertainty (95 % interval)	A	thin line	position (extent)	avoids within-the-bar bias
success per seed (binary)	B	cell	luminance (filled / empty)	2 levels → 1 luminance step
baseline→candidate change (nominal, 4)	B	row band	spatial region	top categorical channel
attribute under baseline and candidate (ratio)	B	dumbbell	position x (symlog if long-tailed)	aligned shift, no crossing
pusher / block or ball / plan (interval, px)	C	line, point, polygon	position (x, y)	spatial data, true geometry
executed / discarded / computing	C	line, point	line style + fill	separable, keeps hue for identity
think, act intervals (time)	D	bar	position + length on time axis	alternation vs overlap visible
movement per step; boundary discontinuity (ratio, px)	D, C	grey area + magenta bar; segment	height on a common baseline + magenta hue	a jump is read against normal motion
coverage or distance to the ball (ratio)	D	line	position y	baseline vs candidate on a common scale
scene in the picture vs. now (interval, px)	C	dotted outline, arrow	position (x, y), same map	gap = outdated plan, read in place
chunk provenance (picture → think → act)	D	○ + outlines + link	position on time axis	connects one chunk's three moments
use of the 16 planned moves (ordinal, 3 levels)	D	16 cells	position (index) + luminance	skipped / executed / discarded at a glance

Hue budget — one meaning per hue

■ Baseline ■ Candidate

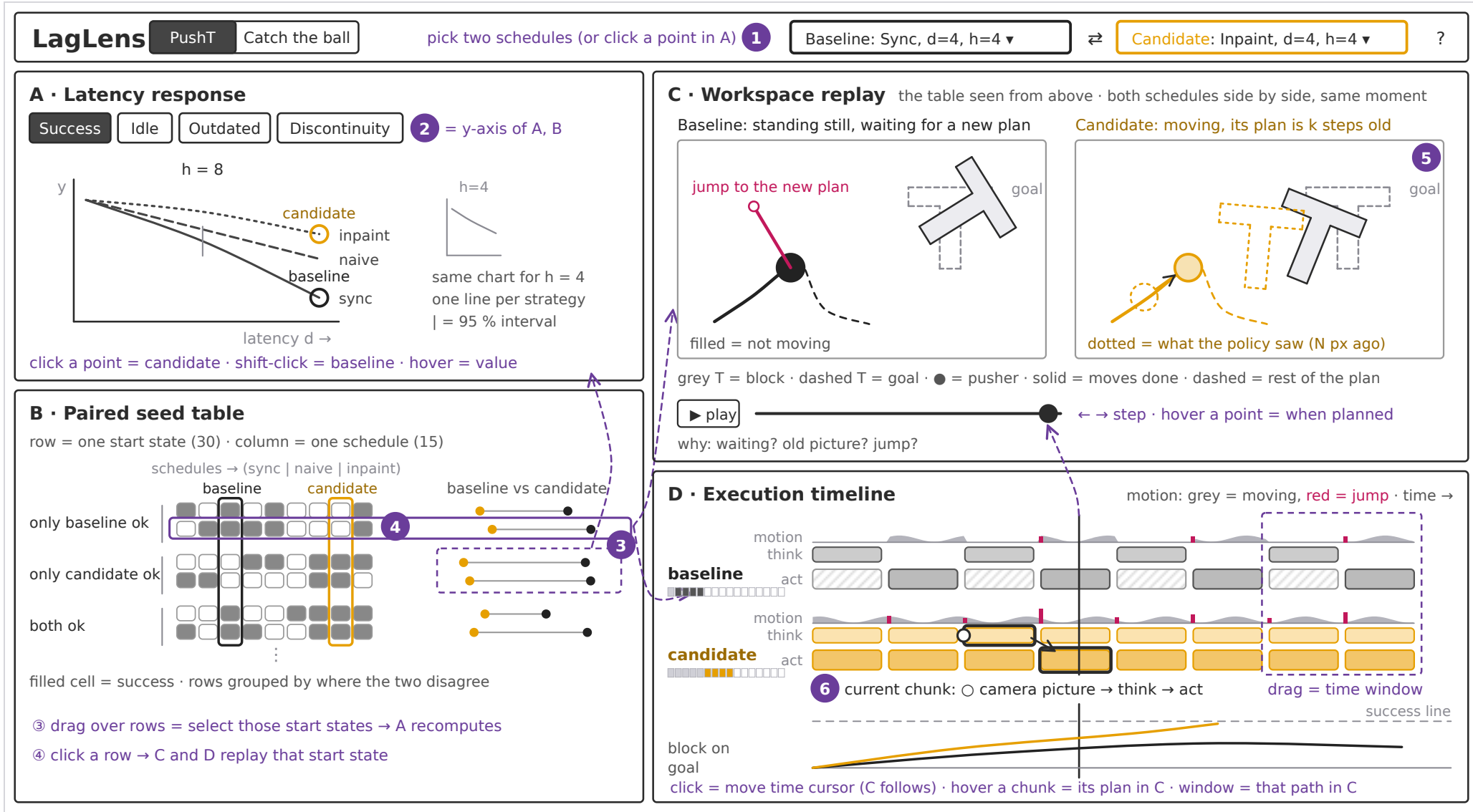
● Sync ■ Naive async ◆ Inpaint

■ Discontinuity (views C, D) grey: everything else

Colour-blind-safe hues, checked by simulating colour-vision deficiency: baseline vs candidate ΔE ≥ 48; strategy hues ≥ 9 plus redundant shape.

Effectiveness ranking used (most important attribute → most effective channel):
magnitude: position on common scale > length > angle/area > luminance
identity: spatial region > hue > shape

Scalability: per task A 15 points · B 450 cells + 30 dumbbells · C/D one episode (≤ 300 steps, ≤ 75 chunks) — SVG at interactive rates.



1. **Pick the task, a baseline and a candidate** schedule: in the header, or click / shift-click a mark in A.
2. **Choose the question** (Success · Idle · Outdated · Discontinuity): A's y-axis and B's dumbbells follow.
3. **Find flipped start states** in B; drag over the dumbbells to select some, and A recomputes on them.
4. **Click a start state**: view C replays it under both schedules side by side (▶ play, ← →).
5. **See why** in C: what the policy saw (dotted) vs. where the robot is now, and the jump at each chunk switch.
6. **See when** in D: the current chunk is traced picture → think → act, a strip shows which of its 16 moves were used; drag a window to highlight its path in C.

Low-fidelity sketch: marks are schematic placeholders, not data. d3.js v7 only.